

# Commission Calculator

---

- [Overview](#)
    - [Class Diagrams](#)
    - [Sequence Diagrams](#)
    - [Use Cases](#)
- 

## Overview

# Sales Commission Calculator

---

## Introduction

A Sales Commission Calculator is a comprehensive internal software platform specifically designed for sales teams to autonomously manage, calculate, track, and optimize their commission structures and earnings. This specialized tool empowers sales organizations to take full control of their compensation processes while providing unprecedented transparency and motivation for individual contributors and sales managers alike.

## Core Functionality

### Intelligent Multi-Tier Calculation Engine

The platform's sophisticated calculation engine processes complex commission scenarios in real-time, handling multiple compensation variables simultaneously including individual deal values, product category multipliers, territory performance bonuses, team collaboration splits, seasonal adjustments, and time-sensitive incentives. The system seamlessly manages accelerators and decelerators across multiple quota periods, processes retroactive adjustments for deal modifications, handles partial payments and installment-based commissions, and accommodates complex scenarios like deal cancellations, refunds, customer upgrades, and cross-selling bonuses.

The calculation engine supports unlimited commission plan variations within a single organization, allowing different teams, roles, or regions to operate under completely different compensation structures. It processes calculations down to fractional cents while maintaining perfect accuracy across thousands of transactions, automatically handling rounding rules, currency conversions, and tax implications based on configured business rules.

### Advanced Commission Plan Architecture

Sales teams can design and implement sophisticated compensation structures through an intuitive visual plan builder that requires no technical expertise. The platform supports base salary configurations, draw against commission arrangements, commission caps and floors, multi-tier accelerator programs, team-based bonus pools, individual performance multipliers, and special promotional incentives (SPIFs).

Plan architects can create conditional logic that triggers different commission rates based on deal characteristics, customer types, product mixes, or sales cycle lengths. The system maintains complete version

control of all plan modifications, allowing teams to test new structures in sandbox environments before implementation while preserving historical accuracy for all past calculations.

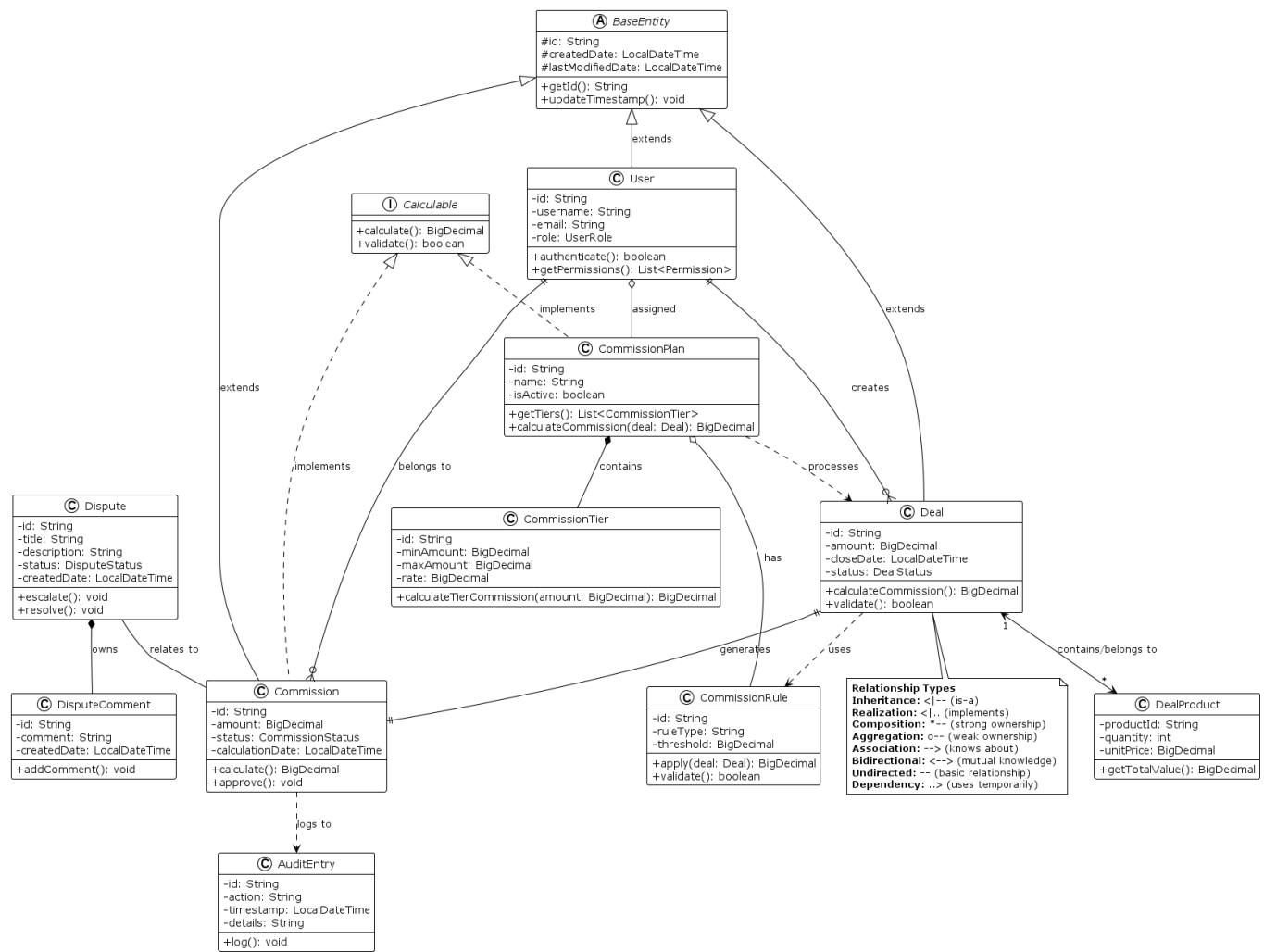
## Real-Time Performance Intelligence

Beyond traditional quota tracking, the platform provides comprehensive performance analytics that help sales professionals understand and optimize their earning potential. Individual dashboards display current earnings progression, projected monthly and quarterly income based on pipeline velocity, quota attainment forecasts with confidence intervals, and detailed breakdowns of commission sources.

The system tracks leading performance indicators including deal velocity through pipeline stages, conversion rates by opportunity type, average deal sizes across different customer segments, and seasonal performance patterns. Sales reps receive personalized insights about which activities and deal types generate the highest commission returns, enabling them to optimize their sales strategies for maximum earnings.

## Class Diagrams

### \Class Diagrams



## Sales Commission Calculator - Model Class Diagram Description

# Overview

The Sales Commission Calculator class diagram represents the core domain model for a comprehensive commission management system. The design follows object-oriented principles with clear separation of concerns, proper inheritance hierarchies, and well-defined relationships between entities that handle users, deals, commissions, disputes, and audit trails.

## Core Classes

### User Management

#### User

**Purpose:** Represents system users including sales representatives, managers, and administrators.

**Attributes:**

- `id: String` - Unique identifier for the user
- `username: String` - Login username
- `email: String` - User's email address for notifications
- `role: UserRole` - Enumerated role determining system permissions

**Methods:**

- `authenticate(): boolean` - Validates user credentials
- `getPermissions(): List<Permission>` - Returns role-based permissions

### Deal Management

#### Deal

**Purpose:** Represents sales transactions that generate commissions.

**Attributes:**

- `id: String` - Unique deal identifier
- `amount: BigDecimal` - Total deal value using precise decimal arithmetic
- `closeDate: LocalDateTime` - When the deal was completed
- `status: DealStatus` - Current state of the deal (e.g., PENDING, CLOSED, CANCELLED)

**Methods:**

- `calculateCommission(): BigDecimal` - Computes commission based on deal value
- `validate(): boolean` - Ensures deal data integrity

#### DealProduct

**Purpose:** Represents individual products within a deal for detailed commission calculations.

**Attributes:**

- `productId: String` - Reference to product catalog

- `quantity: int` - Number of units sold
- `unitPrice: BigDecimal` - Price per individual unit

#### Methods:

- `getTotalValue(): BigDecimal` - Calculates total value for this product line

## Commission Management

### Commission

**Purpose:** Represents calculated commission payments for sales representatives.

#### Attributes:

- `id: String` - Unique commission identifier
- `amount: BigDecimal` - Calculated commission amount
- `status: CommissionStatus` - Payment status (e.g., PENDING, APPROVED, PAID)
- `calculationDate: LocalDateTime` - When commission was calculated

#### Methods:

- `calculate(): BigDecimal` - Computes final commission amount
- `approve(): void` - Approves commission for payment

### CommissionPlan

**Purpose:** Defines the structure and rules for commission calculations.

#### Attributes:

- `id: String` - Plan identifier
- `name: String` - Descriptive name for the commission plan
- `isActive: boolean` - Whether this plan is currently in use

#### Methods:

- `getTiers(): List<CommissionTier>` - Returns all commission tiers
- `calculateCommission(deal: Deal): BigDecimal` - Applies plan to specific deal

### CommissionTier

**Purpose:** Defines tiered commission rates based on deal amounts.

#### Attributes:

- `id: String` - Tier identifier
- `minAmount: BigDecimal` - Minimum deal amount for this tier
- `maxAmount: BigDecimal` - Maximum deal amount for this tier
- `rate: BigDecimal` - Commission percentage for this tier

#### Methods:

- `calculateTierCommission(amount: BigDecimal): BigDecimal` - Applies tier rate to amount

## CommissionRule

**Purpose:** Implements specific business rules for commission calculations.

### Attributes:

- `id: String` - Rule identifier
- `ruleType: String` - Type of rule (e.g., BONUS, PENALTY, MULTIPLIER)
- `threshold: BigDecimal` - Amount threshold for rule application

### Methods:

- `apply(deal: Deal): BigDecimal` - Applies rule logic to deal
- `validate(): boolean` - Ensures rule configuration is valid

## Dispute Management

### Dispute

**Purpose:** Handles commission disputes raised by sales representatives.

### Attributes:

- `id: String` - Dispute identifier
- `title: String` - Brief description of the dispute
- `description: String` - Detailed dispute explanation
- `status: DisputeStatus` - Current dispute state (e.g., OPEN, UNDER\_REVIEW, RESOLVED)
- `createdDate: LocalDateTime` - When dispute was created

### Methods:

- `escalate(): void` - Escalates dispute to higher authority
- `resolve(): void` - Marks dispute as resolved

### DisputeComment

**Purpose:** Stores comments and discussion history for disputes.

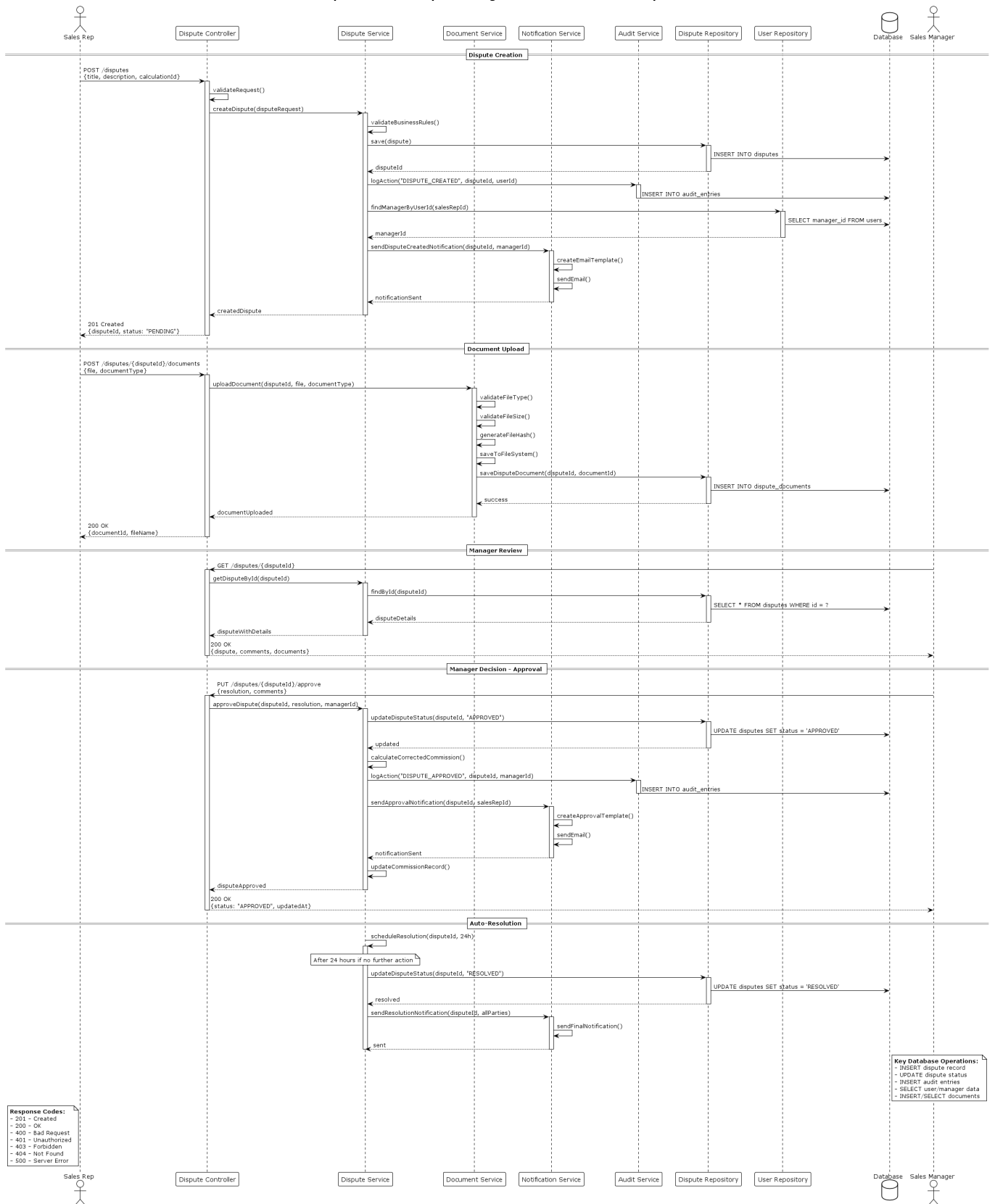
### Attributes:

- `id: String` - Comment identifier
- `comment: String` - Comment text content
- `createdDate: LocalDateTime`

## Sequence Diagrams

\Sequence Diagrams

Dispute Service - Sequence Diagram: Create and Process Dispute



# Dispute Service - Sequence Diagram Description

## Overview

The Dispute Service sequence diagram illustrates the complete workflow for creating, processing, and resolving commission disputes. The process involves multiple system components working together to handle

dispute lifecycle management with proper validation, auditing, and notifications.

## Participants

### Actors

- **Sales Rep:** The user who initiates and manages disputes
- **Sales Manager:** The approving authority who reviews and decides on disputes

### System Components

- **Dispute Controller:** REST API endpoint handler for dispute operations
- **Dispute Service:** Core business logic for dispute processing
- **Document Service:** Handles file uploads and document management
- **Notification Service:** Manages email notifications and communications
- **Audit Service:** Records all system activities for compliance tracking
- **Dispute Repository:** Data access layer for dispute-related operations
- **User Repository:** Data access layer for user and manager information
- **Database:** Persistent storage for all system data

## Process Flow

### Phase 1: Dispute Creation

#### Initial Request

1. **Sales Rep Initiates:** Sales representative sends POST request to `/disputes` endpoint with dispute details including title, description, and calculationId
2. **Request Validation:** Controller validates the incoming request format and required fields
3. **Business Logic Processing:** Dispute Service performs business rule validation to ensure dispute meets system criteria

#### Data Persistence

4. **Database Storage:** Repository saves the new dispute record to the database with initial "PENDING" status
5. **Audit Logging:** Audit Service creates an audit entry recording the dispute creation action
6. **Manager Identification:** System queries User Repository to find the appropriate manager for the sales representative

#### Notification Process

7. **Notification Generation:** Notification Service creates and sends email notification to the identified manager
8. **Response:** Controller returns HTTP 201 Created status with dispute ID and initial status to the sales representative

### Phase 2: Document Upload

## File Handling

1. **Document Submission:** Sales representative uploads supporting documents via POST to `/disputes/{disputeId}/documents`
2. **File Validation:** Document Service performs comprehensive validation including file type checking, size limits, and security scanning
3. **File Processing:** System generates file hash for integrity verification and saves file to secure file system

## Document Association

4. **Database Linking:** Repository creates association between the uploaded document and the specific dispute
5. **Confirmation:** System returns document ID and filename confirmation to the sales representative

## Phase 3: Manager Review

### Information Retrieval

1. **Dispute Access:** Sales manager requests dispute details via GET `/disputes/{disputeId}`
2. **Data Aggregation:** Service retrieves complete dispute information including comments and associated documents
3. **Manager Interface:** System presents comprehensive dispute view to manager for informed decision-making

## Phase 4: Manager Decision - Approval Process

### Approval Workflow

1. **Decision Submission:** Manager submits approval via PUT `/disputes/{disputeId}/approve` with resolution details and comments
2. **Status Update:** Repository updates dispute status to "APPROVED" in the database
3. **Commission Calculation:** Service calculates corrected commission amounts based on dispute resolution

### Audit and Notification

4. **Activity Logging:** Audit Service records the approval action with manager identification
5. **Stakeholder Notification:** Notification Service sends approval confirmation to the original sales representative
6. **Commission Update:** System updates the commission record to reflect approved changes

## Phase 5: Auto-Resolution

### Automated Closure

1. **Schedule Resolution:** System schedules automatic resolution after 24-hour waiting period
2. **Status Finalization:** After waiting period expires, Repository updates dispute status to "RESOLVED"
3. **Final Notification:** Notification Service sends resolution confirmation to all involved parties



# Technical Implementation Details

## Database Operations

The system performs several key database operations throughout the process:

- **INSERT Operations:** New dispute records, audit entries, and document associations
- **UPDATE Operations:** Dispute status changes and commission record modifications
- **SELECT Operations:** User and manager data retrieval, dispute information queries

## HTTP Response Codes

The API uses standard HTTP status codes for communication:

- **201 Created:** Successful dispute creation
- **200 OK:** Successful operations and data retrieval
- **400 Bad Request:** Invalid request format or missing required fields
- **401 Unauthorized:** Authentication failure
- **403 Forbidden:** Insufficient permissions for requested action
- **404 Not Found:** Requested resource does not exist
- **500 Server Error:** Internal system errors

## Security and Validation

### Request Validation

- Input sanitization and format validation at controller level
- Business rule enforcement at service layer
- File type and size validation for document uploads

### Audit Trail

- Comprehensive logging of all dispute-related actions
- User identification tracking for all operations
- Timestamp recording for compliance requirements

## Notification System

### Email Templates

- Dispute creation notifications for managers
- Approval confirmations for sales representatives
- Resolution notifications for all stakeholders

### Delivery Mechanism

- Asynchronous email processing to prevent system delays
- Delivery confirmation tracking
- Template-based messaging for consistency

## Error Handling

The system includes robust error handling at multiple levels:

- **Controller Level:** Input validation and HTTP error responses
- **Service Level:** Business logic validation and exception handling
- **Repository Level:** Database constraint enforcement and transaction management
- **Integration Level:** External service failure handling and retry logic

## Performance Considerations

### Asynchronous Operations

- File upload processing occurs independently of main workflow
- Email notifications are sent asynchronously to prevent blocking
- Audit logging is performed in parallel with main operations

### Data Optimization

- Database queries are optimized for dispute retrieval operations
- Document storage uses efficient file system organization
- Audit data is structured for fast compliance reporting

## Business Rules

### Dispute Validation

- Commission calculation validation against original records
- Business rule compliance checking before dispute creation
- Manager assignment based on organizational hierarchy

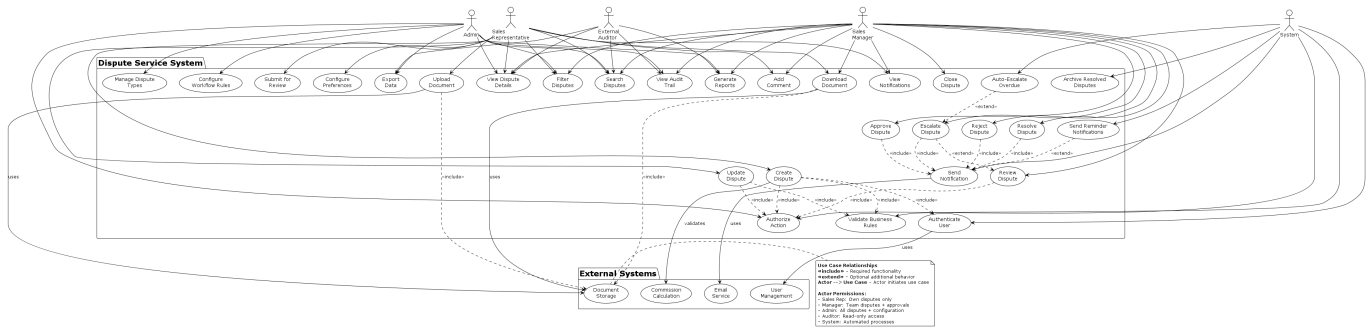
### Approval Authority

- Manager verification before approval processing
- Commission correction calculations with proper validation
- Automatic resolution timing based on business requirements

This sequence diagram represents a comprehensive dispute management system that ensures proper validation, auditing, and stakeholder communication throughout the entire dispute lifecycle.

## Use Cases

\Use Cases



# Dispute Service System - Use Case Diagram Description

## Overview

The Dispute Service System is a comprehensive platform designed to manage commission disputes through a structured workflow involving multiple actors with different levels of access and responsibilities.

## Actors

### Primary Actors

- **Sales Representative:** Front-line users who create and manage their own disputes
- **Sales Manager:** Supervisory role with approval authority over team disputes
- **Admin:** System administrators with full configuration and management capabilities
- **External Auditor:** Read-only access for compliance and audit purposes

### System Actor

- **System:** Automated processes that handle notifications, escalations, and business rule validation

## Core Functionality

### Dispute Management (Row 1)

The system provides comprehensive dispute lifecycle management:

- **Create Dispute:** Sales representatives can initiate new disputes
- **View Dispute Details:** All actors can view dispute information based on their permissions
- **Update Dispute:** Modify dispute information and status
- **Add Comment:** Collaborative commenting system for all stakeholders
- **Upload Document:** Attach supporting documentation to disputes
- **Download Document:** Retrieve dispute-related documents

### Search and Workflow (Row 2)

Advanced search and review capabilities:

- **Search Disputes:** Query disputes using various criteria
- **Filter Disputes:** Apply filters to narrow down dispute lists

- **Submit for Review:** Sales reps submit disputes to management
- **Review Dispute:** Managers evaluate submitted disputes
- **Approve Dispute:** Management approval of valid disputes
- **Reject Dispute:** Management rejection of invalid disputes

## Advanced Workflow (Row 3)

Escalation and resolution processes:

- **Escalate Dispute:** Manual escalation for complex cases
- **Resolve Dispute:** Mark disputes as resolved
- **Close Dispute:** Final closure of dispute cases
- **Send Notification:** System-generated notifications
- **View Notifications:** Users can review their notifications
- **Configure Preferences:** Personal notification and display settings

## Administrative Functions (Row 4)

System configuration and reporting:

- **Manage Dispute Types:** Admin configuration of dispute categories
- **Configure Workflow Rules:** Setup of business process rules
- **Generate Reports:** Create various analytical reports
- **Export Data:** Extract dispute data for external analysis
- **View Audit Trail:** Complete activity logging for compliance
- **Authenticate User:** Security validation for system access

## Automated System Functions (Row 5)

System-driven processes:

- **Auto-Escalate Overdue:** Automatic escalation of stale disputes
- **Send Reminder Notifications:** Automated reminder system
- **Archive Resolved Disputes:** Automatic archiving of closed cases
- **Validate Business Rules:** System enforcement of business logic
- **Authorize Action:** Permission validation for user actions

# External System Integration

The Dispute Service integrates with four external systems:

## Commission Calculation System

- Validates dispute data against commission calculations
- Ensures disputes are based on accurate commission information

## Email Service

- Powers the notification system
- Sends automated emails for workflow events

## User Management System

- Handles user authentication
- Manages user roles and permissions

## Document Storage System

- Stores uploaded dispute documents
- Provides secure document retrieval

## Relationship Types

### Include Relationships (<>)

Required functionality that must be performed:

- **Authentication:** All dispute creation and updates require user authentication
- **Authorization:** Actions require proper permission validation
- **Business Rule Validation:** Dispute operations must comply with business rules
- **Document Storage:** File uploads and downloads use external document storage

### Extend Relationships (<>)

Optional additional behaviors:

- **Auto-Escalation:** System can automatically escalate overdue disputes
- **Reminder Notifications:** System extends basic notifications with reminders
- **Manual Escalation:** Reviews can be extended with escalation processes

## Actor Permissions

### Sales Representative

- **Scope:** Own disputes only
- **Permissions:** Create, view, update own disputes; add comments; upload/download documents; search and filter; submit for review; configure personal preferences

### Sales Manager

- **Scope:** Team disputes plus approval authority
- **Permissions:** View team disputes; review, approve, reject disputes; escalate complex cases; resolve and close disputes; generate reports; view audit trails

### Admin

- **Scope:** All disputes plus system configuration
- **Permissions:** Full system access; manage dispute types; configure workflow rules; generate comprehensive reports; export data; view complete audit trails; authorize system-wide actions

### External Auditor

- **Scope:** Read-only access for compliance
- **Permissions:** View disputes; search and filter; generate reports; export data; view audit trails (no modification rights)

## System

- **Scope:** Automated processes
- **Functions:** Send notifications; auto-escalate overdue disputes; send reminders; archive resolved cases; validate business rules; authenticate users; authorize actions

## Workflow Summary

The typical dispute workflow follows this pattern:

1. Sales Rep creates dispute (with authentication and validation)
2. Sales Rep submits dispute for review
3. Manager reviews and either approves/rejects (triggering notifications)
4. If needed, dispute can be escalated
5. Approved disputes are resolved and eventually closed
6. System automatically handles reminders and archives completed disputes
7. All activities are logged for audit purposes